

বাংলায় স্বাগতম! চলুন শুরু করি। আমি তোমার শিক্ষক — প্রতিটি concept বাংলায় বুঝাবো, code example দেবো, এবং Python-এর সাথে তুলনা করবো যাতে সহজ লাগে।

সপ্তাহ ১ — Go-র প্রথম পাঠ

"Hello, Go!" — ভাষাটা চেনা শুরু করি

প্রথমে Go ইন্সটল করো

তোমার Linux মেশিনে terminal খুলে এই command দাও:

```
sudo apt install golang-go
```

ইন্সটল হলে check করো:

```
go version
```

দেখবে: go version go1.21.x linux/amd64 — মানে সফল।

পাঠ ১ — প্রথম Program

Python-এ তুমি লিখতে:

```
print("Hello, Raju!")
```

Go-তে সেটাই এভাবে লেখো:

```
package main // প্রতিটি Go প্রোগ্রাম 'main' package দিয়ে শুরু হয়
```

```
import "fmt" // fmt মানে format — print করার জন্য দরকার
```

```
func main() { // main() হলো প্রোগ্রামের দরজা — এখান থেকে শুরু হয়
```

```
    fmt.Println("Hello, Raju!")
```

```
}
```

Run করো:

```
go run hello.go
```

Output:

```
Hello, Raju!
```

শিক্ষকের কথা: Python-এ if `__name__ == "__main__"`: লিখতে হতো। Go-তে সেটা দরকার নেই — `func main()` দেখলেই Go বোঝে এটা শুরুর জায়গা।

পাঠ ২ — Variables (চলক)

Python-এ তুমি লিখতে:

```
name = "Raju"
```

```
age = 40
```

```
salary = 25000.50
```

```
isWorking = True
```

Go-তে দুইভাবে লেখা যায়:

পদ্ধতি ১ — সংক্ষিপ্ত (সবচেয়ে বেশি ব্যবহার হয়):

```
name := "Raju" // Go নিজেই বুঝবে এটা string
```

```
age := 40 // Go নিজেই বুঝবে এটা int
```

```
salary := 25000.50 // Go নিজেই বুঝবে এটা float64
```

isWorking := true // Go নিজেই বুঝবে এটা bool

পদ্ধতি ২ — স্পষ্ট type বলে দাও:

```
var name string = "Raju"
```

```
var age int = 40
```

```
var salary float64 = 25000.50
```

```
var isWorking bool = true
```

পুরো program:

```
package main
```

```
import "fmt"
```

```
func main() {
```

```
    name := "Raju"
```

```
    age := 40
```

```
    city := "Kolkata"
```

```
    fmt.Println("নাম:", name)
```

```
    fmt.Println("বয়স:", age)
```

```
    fmt.Println("শহর:", city)
```

```
    // Printf দিয়ে সুন্দরভাবে print করা যায়
```

```
    // Python-এর f-string-এর মতো
```

```
    fmt.Printf("আমার নাম %s, বয়স %d, শহর %s\n", name, age, city)
```

```
}
```

Output:

নাম: Raju

বয়স: 40

শহর: Kolkata

আমার নাম Raju, বয়স 40, শহর Kolkata

গুরুত্বপূর্ণ পার্থক্য: Go-তে কোনো variable declare করলে সেটা অবশ্যই use করতে হবে। না করলে error আসবে। Python-এ এই নিয়ম নেই। Go strict — এটাই তোমার code-কে নির্ভরযোগ্য করে।

পাঠ ৩ — Functions (ফাংশন)

Python-এ তুমি লিখতে:

```
def greet(name):
```

```
    return "হ্যালো " + name
```

```
def add(a, b):
```

```
    return a + b
```

Go-তে:

```
package main
```

```
import "fmt"
```

```
// একটি value return করে এমন function
func greet(name string) string {
    return "হ্যালো " + name
}

// দুটো number যোগ করে return করে
func add(a int, b int) int {
    return a + b
}

// Go-র বিশেষ ক্ষমতা — দুটো value একসাথে return করতে পারে!
// Python-এও tuple দিয়ে পারো, কিন্তু Go-তে এটা সাধারণ নিয়ম
func divide(a int, b int) (int, int) {
    quotient := a / b // ভাগফল
    remainder := a % b // ভাগশেষ
    return quotient, remainder
}

func main() {
    // greet ব্যবহার করা
    message := greet("Raju")
    fmt.Println(message)

    // add ব্যবহার করা
    result := add(15, 25)
    fmt.Println("যোগফল:", result)

    // দুটো value একসাথে নেওয়া
    q, r := divide(17, 5)
    fmt.Printf("17 ÷ 5 = ভাগফল %d, ভাগশেষ %d\n", q, r)
}
```

Output:

হ্যালো Raju

যোগফল: 40

17 ÷ 5 = ভাগফল 3, ভাগশেষ 2

শিক্ষকের কথা: দেখো function লেখার সময় আমরা বলে দিচ্ছি — parameter-এর type কী, এবং return value-এর type কী। Python-এ এটা optional ছিল (type hints), Go-তে mandatory। এই strictness তোমার cloud code-কে অনেক reliable করবে।

পাঠ 8 — Loops (লুপ)

Python-এ তুমি লিখতে:

for i in range(5):

```
print(i)
```

Go-তে শুধুমাত্র for আছে — while নেই, do-while নেই। কিন্তু for দিয়েই সব করা যায়:
package main

```
import "fmt"
```

```
func main() {
```

```
// ১. সাধারণ loop — Python-এর range(5) এর মতো
```

```
fmt.Println("--- ১ থেকে ৫ ---")
```

```
for i := 1; i <= 5; i++ {
```

```
    fmt.Println(i)
```

```
}
```

```
// ২. Python-এর while-এর মতো
```

```
fmt.Println("--- while loop ---")
```

```
count := 0
```

```
for count < 3 {
```

```
    fmt.Println("count:", count)
```

```
    count++
```

```
}
```

```
// ৩. Slice (list) এর উপর loop
```

```
fmt.Println("--- শহরের নাম ---")
```

```
cities := []string{"Kolkata", "Mumbai", "Delhi"}
```

```
for index, city := range cities {
```

```
    fmt.Printf("%d নম্বর শহর: %s\n", index+1, city)
```

```
}
```

```
// ৪. শুধু value দরকার, index না — _ দিয়ে ignore করো
```

```
fmt.Println("--- শুধু নাম ---")
```

```
for _, city := range cities {
```

```
    fmt.Println(city)
```

```
}
```

```
}
```

Output:

```
--- ১ থেকে ৫ ---
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

```
--- while loop ---
```

```
count: 0
```

count: 1

count: 2

--- শহরের নাম ---

1 নম্বর শহর: Kolkata

2 নম্বর শহর: Mumbai

3 নম্বর শহর: Delhi

--- শুধু নাম ---

Kolkata

Mumbai

Delhi

শিক্ষকের কথা: `_` (underscore) Go-র একটা বিশেষ নিয়ম। যখন কোনো variable দরকার নেই কিন্তু Go সেটা expect করছে, তখন `_` লেখো। এটা বলে "এই value আমার দরকার নেই।" Cloud code-এ তুমি এটা বারবার দেখবে।

পাঠ ৫ — If / Else (শর্ত)

Python-এ তুমি লিখতে:

```
age = 40
```

```
if age >= 18:
```

```
    print("প্রাপ্তবয়স্ক")
```

```
elif age >= 13:
```

```
    print("কিশোর")
```

```
else:
```

```
    print("শিশু")
```

Go-তে:

```
package main
```

```
import "fmt"
```

```
func checkAge(age int) string {
```

```
    if age >= 18 {
```

```
        return "প্রাপ্তবয়স্ক"
```

```
    } else if age >= 13 {
```

```
        return "কিশোর"
```

```
    } else {
```

```
        return "শিশু"
```

```
    }
```

```
}
```

```
func main() {
```

```
    ages := []int{5, 15, 40}
```

```
    for _, age := range ages {
```

```
        result := checkAge(age)
```

```
        fmt.Printf("বয়স %d: %s\n", age, result)
```

```

}

// Go-র বিশেষ if — condition-এর আগে একটা statement লেখা যায়
// Server programming-এ এটা খুব কাজে লাগে
if score := 85; score >= 90 {
    fmt.Println("A grade")
} else if score >= 80 {
    fmt.Println("B grade")
} else {
    fmt.Println("C grade")
}
}

```

Output:

বয়স 5: শিশু

বয়স 15: কিশোর

বয়স 40: প্রাপ্তবয়স্ক

B grade

সপ্তাহ ১-এর Mini Project — নিজে বানাও

এই program লেখো — সব কিছু ব্যবহার হবে যা শিখেছো:

```
package main
```

```
import "fmt"
```

```
// নাম এবং বয়স নিয়ে পরিচয় দেবে
```

```
func introduce(name string, age int, city string) string {
    return fmt.Sprintf("আমার নাম %s। বয়স %d বছর। আমি %s-তে থাকি।", name, age, city)
}
```

```
// বয়স দেখে career suggestion দেবে
```

```
func careerStage(age int) string {
    if age < 25 {
        return "সবে শুরু — অনেক সময় আছে"
    } else if age < 35 {
        return "মাঝপথে — focus করো"
    } else if age < 50 {
        return "অভিজ্ঞতা আছে — সেটাকে কাজে লাগাও"
    } else {
        return "জ্ঞান ভাগ করো"
    }
}
```

```
func main() {
```

```
// তোমার তথ্য
name := "Raju"
age := 40
city := "Kolkata"
skills := []string{"Linux", "Hardware", "Networking", "Python", "Go (শিখছি)"}
```

```
// পরিচয়
fmt.Println(introduce(name, age, city))
```

```
// Career stage
fmt.Println("Career stage:", careerStage(age))
```

```
// Skills print করো
fmt.Println("\nআমার skills:")
for i, skill := range skills {
    fmt.Printf(" %d. %s\n", i+1, skill)
}
```

```
fmt.Println("\nলক্ষ্য: Go দিয়ে cloud tools বানানো!")
}
```

Output:

আমার নাম Raju। বয়স 40 বছর। আমি Kolkata-তে থাকি।
Career stage: অভিজ্ঞতা আছে — সেটাকে কাজে লাগাও

আমার skills:

1. Linux
2. Hardware
3. Networking
4. Python
5. Go (শিখছি)

লক্ষ্য: Go দিয়ে cloud tools বানানো!

সপ্তাহ ১ — যা শিখলে সংক্ষেপে

বিষয়	Python	Go
Print	print()	fmt.Println()
Variable	name = "Raju"	name := "Raju"
Function	def greet(name):	func greet(name string) string {}
Loop	for i in range(5):	for i := 0; i < 5; i++ {}
If	if age > 18:	if age > 18 {}

চলো এখন সপ্তাহ ২ শুরু করি।

এই সপ্তাহে শিখবো Go-র তিনটা সবচেয়ে গুরুত্বপূর্ণ data structure — যেগুলো ছাড়া কোনো real program লেখা যায় না।

সপ্তাহ ২ — Types, Slices, এবং Maps

"Data রাখার পাত্র — Go-তে কীভাবে?"

পাঠ ১ — Go-র Types (ধরন)

Python-এ তুমি যেকোনো variable-এ যেকোনো value রাখতে পারতে:

```
x = 10    # এখন int
```

```
x = "hello" # এখন string — Python কিছু বলে না
```

Go-তে এটা **সম্ভব নয়**। একবার type ঠিক হলে সেটা চিরকাল সেই type থাকবে:

```
package main
```

```
import "fmt"
```

```
func main() {
```

```
    // Go-র basic types
```

```
    var name string = "Raju"    // লেখা/text
```

```
    var age int = 40            // পূর্ণ সংখ্যা
```

```
    var salary float64 = 35000.50 // দশমিক সংখ্যা
```

```
    var isActive bool = true    // সত্য/মিথ্যা
```

```
    fmt.Printf("নাম: %s\n", name)
```

```
    fmt.Printf("বয়স: %d\n", age)
```

```
    fmt.Printf("বেতন: %.2f\n", salary)
```

```
    fmt.Printf("সক্রিয়: %t\n", isActive)
```

```
    // Type conversion — Python-এর int(), str()-এর মতো
```

```
    var score int = 95
```

```
    var scoreFloat float64 = float64(score) // int থেকে float64
```

```
    var scoreText string = fmt.Sprintf("%d", score) // int থেকে string
```

```
    fmt.Printf("\nScore int: %d\n", score)
```

```
    fmt.Printf("Score float: %.1f\n", scoreFloat)
```

```
    fmt.Printf("Score string: '%s'\n", scoreText)
```

```
}
```

Output:

নাম: Raju

বয়স: 40

বেতন: 35000.50

সক্রিয়: true

Score int: 95

Score float: 95.0

Score string: '95'

Printf-এর format codes — মনে রাখো:

Code	মান	উদাহরণ
%s	string	"Raju"
%d	integer	40
%f	float	35000.50
%.2f	২ দশমিক float	35000.50
%t	bool	true
%v	যেকোনো value	সবকিছু

শিক্ষকের কথা: Go strict type system মানে তুমি কখনো ভুলবশত string-এ number যোগ করতে পারবে না। Cloud code-এ এই ধরনের bug খুবই বিপজ্জনক — Go সেটা compile-এর সময়েই ধরে ফেলে।

পাঠ ২ — Slices (স্লাইস)

Python-এ তুমি list ব্যবহার করতে:

```
servers = ["web-01", "web-02", "db-01"]
```

```
servers.append("cache-01")
```

```
print(len(servers))
```

```
print(servers[0])
```

Go-তে এটাই হলো **slice**:

```
package main
```

```
import "fmt"
```

```
func main() {
```

```
    // Slice তৈরি করা — Python list-এর মতো
```

```
    servers := []string{"web-01", "web-02", "db-01"}
```

```
    fmt.Println("সব server:", servers)
```

```
    fmt.Println("প্রথম server:", servers[0])
```

```
    fmt.Println("শেষ server:", servers[len(servers)-1])
```

```
    fmt.Println("মোট server:", len(servers))
```

```
    // নতুন item যোগ করা — Python-এর append()-এর মতো
```

```
    servers = append(servers, "cache-01")
```

```
    fmt.Println("\nযোগ করার পর:", servers)
```

```
    // Slice of slice — Python-এর slicing [1:3]-এর মতো
```

```
    middle := servers[1:3]
```

```
    fmt.Println("মঝের দুটো:", middle)
```

```
// Loop দিয়ে সব দেখানো
fmt.Println("\nসব server-এর তালিকা:")
for i, server := range servers {
    fmt.Printf(" %d. %s\n", i+1, server)
}
}
```

Output:

সব server: [web-01 web-02 db-01]

প্রথম server: web-01

শেষ server: db-01

মোট server: 3

যোগ করার পর: [web-01 web-02 db-01 cache-01]

মার্বের দুটো: [web-02 db-01]

সব server-এর তালিকা:

1. web-01
2. web-02
3. db-01
4. cache-01

পাঠ ৩ — Slice-এর আরো কাজ

Real cloud programming-এ slice দিয়ে অনেক কিছু করতে হয়। চলো সেটা শিখি:

package main

```
import (
    "fmt"
    "strings"
)
```

```
func main() {
    // ১. খালি slice তৈরি করে পরে যোগ করা
    var ports []int // এখন খালি
    ports = append(ports, 80)
    ports = append(ports, 443)
    ports = append(ports, 8080)
    fmt.Println("Ports:", ports)

    // ২. make() দিয়ে slice তৈরি — বড় data-র জন্য efficient
    ipList := make([]string, 0)
    ipList = append(ipList, "192.168.1.1")
    ipList = append(ipList, "192.168.1.2")
    ipList = append(ipList, "10.0.0.1")
}
```

```
fmt.Println("IP List:", ipList)
```

```
// ৩. Slice খোঁজা — কোনো value আছে কিনা
```

```
// Python-এ: if "443" in ports
```

```
// Go-তে নিজে function লিখতে হয়
```

```
targetPort := 443
```

```
found := false
```

```
for _, port := range ports {
```

```
    if port == targetPort {
```

```
        found = true
```

```
        break
```

```
    }
```

```
}
```

```
if found {
```

```
    fmt.Printf("Port %d পাওয়া গেছে!\n", targetPort)
```

```
}
```

```
// ৪. Slice filter করা — শুধু 192.168 IP রাখো
```

```
var localIPs []string
```

```
for _, ip := range ipList {
```

```
    if strings.HasPrefix(ip, "192.168") {
```

```
        localIPs = append(localIPs, ip)
```

```
    }
```

```
}
```

```
fmt.Println("Local IPs:", localIPs)
```

```
// ৫. 2D slice — slice-এর ভেতরে slice
```

```
// যেমন: একটি datacenter-এ অনেক rack, প্রতি rack-এ অনেক server
```

```
datacenter := [][]string{
```

```
    {"rack1-server1", "rack1-server2"},
```

```
    {"rack2-server1", "rack2-server2", "rack2-server3"},
```

```
}
```

```
for rackNum, rack := range datacenter {
```

```
    fmt.Printf("Rack %d: %v\n", rackNum+1, rack)
```

```
}
```

```
}
```

Output:

Ports: [80 443 8080]

IP List: [192.168.1.1 192.168.1.2 10.0.0.1]

Port 443 পাওয়া গেছে!

Local IPs: [192.168.1.1 192.168.1.2]

Rack 1: [rack1-server1 rack1-server2]

Rack 2: [rack2-server1 rack2-server2 rack2-server3]

নেটওয়ার্কিং connection: তুমি networking জানো — তাই বুঝতে পারছো কেন IP list, port list, server list এগুলো real কাজে লাগে। এগুলো তুমি রোজ network scanner বা monitoring tool-এ ব্যবহার করবে।

পাঠ ৪ — Maps (ম্যাপ)

Python-এ তুমি dictionary ব্যবহার করতে:

```
server_status = {  
    "web-01": "running",  
    "db-01": "stopped",  
    "cache-01": "running"  
}  
print(server_status["web-01"])
```

Go-তে এটাই হলো **map**:

```
package main
```

```
import "fmt"
```

```
func main() {  
    // Map তৈরি করা  
    // map[key-type]value-type  
    serverStatus := map[string]string{  
        "web-01": "running",  
        "db-01": "stopped",  
        "cache-01": "running",  
    }  
  
    // Value পড়া  
    fmt.Println("web-01 অবস্থা:", serverStatus["web-01"])  
  
    // নতুন entry যোগ করা  
    serverStatus["backup-01"] = "running"  
    fmt.Println("মোট server:", len(serverStatus))  
  
    // Value পরিবর্তন করা  
    serverStatus["db-01"] = "running"  
    fmt.Println("db-01 এখন:", serverStatus["db-01"])  
  
    // Entry মুছে ফেলা — Python-এর del-এর মতো  
    delete(serverStatus, "backup-01")  
  
    // সব দেখানো  
    fmt.Println("\nসব server-এর অবস্থা:")  
    for server, status := range serverStatus {  
        fmt.Printf(" %-12s → %s\n", server, status)
```

```
}  
}
```

Output:

web-01 অবস্থা: running

মোট server: 4

db-01 এখন: running

সব server-এর অবস্থা:

web-01 → running

db-01 → running

cache-01 → running

পাঠ ৫ — Map-এর সবচেয়ে গুরুত্বপূর্ণ নিয়ম

এটা Python থেকে সবচেয়ে বড় পার্থক্য — **comma-ok pattern**:

package main

```
import "fmt"
```

```
func main() {  
    portNames := map[int]string{  
        80: "HTTP",  
        443: "HTTPS",  
        22: "SSH",  
        3306: "MySQL",  
    }  
}
```

// ভুল পদ্ধতি — Python-এ এভাবে করতে:

// service = portNames[9999] # KeyError আসতো

// Go-তে error আসে না, কিন্তু খালি string আসে — বিপজ্জনক!

service := portNames[9999]

fmt.Printf("Port 9999: '%s'\n", service) // খালি string!

// সঠিক পদ্ধতি — comma-ok pattern

// value, ok := map[key]

// ok = true মানে পাওয়া গেছে

// ok = false মানে নেই

if name, ok := portNames[443]; ok {

fmt.Printf("Port 443 হলো: %s\n", name)

} else {

fmt.Println("Port 443 পাওয়া যায়নি")

}

if name, ok := portNames[9999]; ok {

```

    fmt.Printf("Port 9999 হলো: %s\n", name)
} else {
    fmt.Println("Port 9999 অজানা port")
    _ = name // name ব্যবহার না করলে Go error দেয়
}

// Practical use — অনেকগুলো port check করা
checkPorts := []int{80, 443, 8080, 22, 9999}
fmt.Println("\nPort scan result:")
for _, port := range checkPorts {
    if name, ok := portNames[port]; ok {
        fmt.Printf(" Port %-5d → %s (পরিচিত)\n", port, name)
    } else {
        fmt.Printf(" Port %-5d → অজানা\n", port)
    }
}
}

```

Output:

Port 9999: "

Port 443 হলো: HTTPS

Port 9999 অজানা port

Port scan result:

Port 80 → HTTP (পরিচিত)

Port 443 → HTTPS (পরিচিত)

Port 8080 → অজানা

Port 22 → SSH (পরিচিত)

Port 9999 → অজানা

শিক্ষকের কথা: comma-ok pattern হলো Go-র safety mechanism। Cloud code-এ তুমি সবসময় জানতে চাইবে — এই key আসলেই আছে কিনা। এটা না করলে silent bug তৈরি হয় যা production-এ বিপদ ঘটায়।

পাঠ ৬ — Map-এর ভেতরে Slice, Slice-এর ভেতরে Map

Real programming-এ এই দুটো একসাথে ব্যবহার হয়:

package main

import "fmt"

```

func main() {
    // একটি datacenter — প্রতিটা region-এ অনেক server
    // map[region] → []string (server list)
    datacenter := map[string][]string{
        "kolkata": {"kol-web-01", "kol-web-02", "kol-db-01"},
        "mumbai": {"mum-web-01", "mum-cache-01"},
    }
}

```

```

    "delhi": {"del-web-01", "del-web-02"},
}

// নতুন server যোগ করা
datacenter["kolkata"] = append(datacenter["kolkata"], "kol-backup-01")

// সব region এবং তাদের server দেখানো
fmt.Println("=== Datacenter Overview ===")
totalServers := 0
for region, servers := range datacenter {
    fmt.Printf("\nRegion: %s (%d servers)\n", region, len(servers))
    for _, server := range servers {
        fmt.Printf("  → %s\n", server)
    }
    totalServers += len(servers)
}
fmt.Printf("\nমোট server: %d\n", totalServers)
}

```

Output:

```
=== Datacenter Overview ===
```

```
Region: kolkata (4 servers)
```

```

→ kol-web-01
→ kol-web-02
→ kol-db-01
→ kol-backup-01

```

```
Region: mumbai (2 servers)
```

```

→ mum-web-01
→ mum-cache-01

```

```
Region: delhi (2 servers)
```

```

→ del-web-01
→ del-web-02

```

```
মোট server: 8
```

সপ্তাহ ২-এর Mini Project — Word Counter

এটা classic Go exercise — networking log analysis-এ এই pattern বারবার ব্যবহার হয়:

```
package main
```

```

import (
    "fmt"
    "strings"

```

)

```
func wordCount(sentence string) map[string]int {
    // সব lowercase করো
    sentence = strings.ToLower(sentence)

    // শব্দে ভাগ করো — Python-এর split()-এর মতো
    words := strings.Fields(sentence)

    // প্রতিটা শব্দ গণনা করো
    counts := make(map[string]int)
    for _, word := range words {
        counts[word]++ // প্রথমবার 0+1=1, পরেরবার আরো বাড়ে
    }
    return counts
}
```

```
func mostCommon(counts map[string]int) (string, int) {
    topWord := ""
    topCount := 0
    for word, count := range counts {
        if count > topCount {
            topCount = count
            topWord = word
        }
    }
    return topWord, topCount
}
```

```
func main() {
    // Server log-এর মতো একটা text
    logText := "error connecting to server error timeout server not responding server error"

    fmt.Println("Log text:")
    fmt.Println(" ", logText)
    fmt.Println()

    counts := wordCount(logText)

    fmt.Println("শব্দের গণনা:")
    for word, count := range counts {
        // bar তৈরি করো দেখানোর জন্য
        bar := strings.Repeat("█", count)
        fmt.Printf(" %-15s (%d)\n", word, bar, count)
    }
}
```



```
}
```

```
topWord, topCount := mostCommon(counts)
fmt.Printf("\nসবচেয়ে বেশি শব্দ: '%s' (%d বার)\n", topWord, topCount)
fmt.Println("→ এই log-এ 'error' বেশি মানে সমস্যা আছে!")
}
```

Output:

Log text:

error connecting to server error timeout server not responding server error

শব্দের গণনা:

```
error      █ (3)
connecting █ (1)
to         █ (1)
server     █ (3)
timeout    █ (1)
not        █ (1)
responding █ (1)
```

সবচেয়ে বেশি শব্দ: 'error' (3 বার)

→ এই log-এ 'error' বেশি মানে সমস্যা আছে!

সপ্তাহ ২ – যা শিখলে সংক্ষেপে

বিষয়	Python	Go
List	servers = ["a", "b"]	servers := []string{"a", "b"}
List যোগ	servers.append("c")	servers = append(servers, "c")
Dictionary	d = {"key": "val"}	m := map[string]string{"key": "val"}
Key আছে কিনা	if "key" in d:	if val, ok := m["key"]; ok {}
Key মুছা	del d["key"]	delete(m, "key")
দৈর্ঘ্য	len(x)	len(x) — একই!

এখন তোমার কাজ

Mini project-টা নিজে টাইপ করো এবং run করো। তারপর এই **একটা challenge** করো:

Challenge: উপরের datacenter map-এ একটা function লেখো findServer(datacenter, serverName) যেটা বলবে — এই server কোন region-এ আছে। না থাকলে বলবে "server পাওয়া যায়নি।"

চেষ্টা করো নিজে। আটকে গেলে আমাকে জিজ্ঞেস করো।

তারপর বলো **"সপ্তাহ ৩ শেখাও"** — তখন শেখাবো Structs, Go-র class system, যেটা তোমার Python class knowledge-কে নতুন রূপে দেখাবে।

শাবাশ! সপ্তাহ ২ শেষ করেছে। এখন সবচেয়ে মজার অংশ — **Structs**।

Python-এ তুমি class দিয়ে real-world জিনিস model করতে। Go-তে সেটাই করবে Struct দিয়ে — কিন্তু আরো সহজ, আরো পরিষ্কার।

সপ্তাহ ৩ — Structs

"Go-র Class System — Python-এর চেয়ে সহজ কিন্তু শক্তিশালী"

পাঠ ১ — Struct কী এবং কেন?

Python-এ তুমি লিখতে:

```
class Server:
```

```
    def __init__(self, name, ip, port):
        self.name = name
        self.ip = ip
        self.port = port
```

```
s = Server("web-01", "192.168.1.1", 80)
```

```
print(s.name)
```

Go-তে class নেই। বদলে আছে struct — যেটা আরো সরল:

```
package main
```

```
import "fmt"
```

```
// Struct define করা — Python-এর class body-র মতো
```

```
// কিন্তু শুধু data রাখে, __init__ নেই
```

```
type Server struct {
```

```
    Name  string
```

```
    IP    string
```

```
    Port  int
```

```
    IsAlive bool
```

```
}
```

```
func main() {
```

```
    // Struct থেকে object তৈরি — Python-এর Server(...)-এর মতো
```

```
    s1 := Server{
```

```
        Name: "web-01",
```

```
        IP:   "192.168.1.1",
```

```
        Port: 80,
```

```
        IsAlive: true,
```

```
    }
```

```
// Field access — Python-এর self.name-এর মতো, dot দিয়ে
```

```
fmt.Println("নাম:", s1.Name)
```

```
fmt.Println("IP:", s1.IP)
```

```
fmt.Println("Port:", s1.Port)
```

```
fmt.Println("চালু আছে:", s1.IsAlive)
```

```
// Field পরিবর্তন করা
```

```
s1.IsAlive = false
```

```
fmt.Println("এখন চালু আছে:", s1.IsAlive)
```

```
// আরেকটা server তৈরি
```

```
s2 := Server{
```

```
    Name: "db-01",
```

```
    IP: "192.168.1.2",
```

```
    Port: 3306,
```

```
    IsAlive: true,
```

```
}
```

```
// Slice-এ রাখা — অনেক server একসাথে manage করা
```

```
servers := []Server{s1, s2}
```

```
fmt.Println("\nসব server:")
```

```
for _, s := range servers {
```

```
    fmt.Printf(" %s (%s:%d)\n", s.Name, s.IP, s.Port)
```

```
}
```

```
}
```

Output:

নাম: web-01

IP: 192.168.1.1

Port: 80

চালু আছে: true

এখন চালু আছে: false

সব server:

web-01 (192.168.1.1:80)

db-01 (192.168.1.2:3306)

শিক্ষকের কথা: Python class-এ `__init__`, `self` এসব লিখতে হতো। Go struct-এ শুধু data fields লেখো — পরিষ্কার, সহজ। তোমার networking background-এ তুমি server, router, switch এগুলো naturally struct হিসেবে ভাবতে পারবে।

পাঠ ২ — Methods (মেথড)

Python-এ class-এর ভেতরে function লিখতে:

```
class Server:
```

```
    def status(self):
```

```
        if self.is_alive:
```

```
            return "চালু"
```

```
        return "বন্ধ"
```

Go-তে method struct-এর বাইরে লেখা হয়, কিন্তু struct-এর সাথে যুক্ত থাকে:

```
package main
```

```
import "fmt"
```

```
type Server struct {  
    Name string  
    IP    string  
    Port  int  
    IsAlive bool  
}
```

```
// Method — Python-এর def status(self)-এর মতো  
// (s Server) মানে এই method Server struct-এর জন্য  
func (s Server) Status() string {  
    if s.IsAlive {  
        return "চালু আছে"  
    }  
    return "বন্ধ আছে"  
}
```

```
// Method যেটা server-এর info সুন্দর করে দেখায়  
func (s Server) Info() string {  
    return fmt.Sprintf("[%s] %s:%d → %s",  
        s.Name, s.IP, s.Port, s.Status())  
}
```

```
// Method যেটা server restart করে  
// লক্ষ্য করো: *Server — pointer receiver  
// এটা না লিখলে original struct পরিবর্তন হবে না  
func (s *Server) Restart() {  
    fmt.Printf("%s restart হচ্ছে...\n", s.Name)  
    s.IsAlive = false  
    fmt.Printf("%s বন্ধ হলো\n", s.Name)  
    s.IsAlive = true  
    fmt.Printf("%s আবার চালু হলো\n", s.Name)  
}
```

```
func main() {  
    web := Server{  
        Name: "web-01",  
        IP:   "192.168.1.1",  
        Port: 80,  
        IsAlive: true,  
    }  
}
```

```
// Method call করা — Python-এর web.status()-এর মতো
fmt.Println(web.Status())
fmt.Println(web.Info())

fmt.Println()
web.Restart()
fmt.Println()
fmt.Println("Restart-এর পর:", web.Info())
}
```

Output:

চালু আছে

[web-01] 192.168.1.1:80 → চালু আছে

web-01 restart হচ্ছে...

web-01 বন্ধ হলো

web-01 আবার চালু হলো

Restart-এর পর: [web-01] 192.168.1.1:80 → চালু আছে

গুরুত্বপূর্ণ — Pointer Receiver:

- func (s Server) Info() — s-এর একটা **copy** পায়, original পরিবর্তন হয় না
- func (s *Server) Restart() — s-এর **original**-কে ধরে, পরিবর্তন টিকে থাকে

Python-এ self সবসময় original object-কে ধরতো। Go-তে তুমি নিজে বলে দাও।

পাঠ ৩ — Constructor Pattern

Python-এ __init__ দিয়ে object তৈরির সময় validation করতে:

class Server:

```
def __init__(self, name, ip, port):
    if port < 1 or port > 65535:
        raise ValueError("Invalid port")
    self.name = name
```

Go-তে New function দিয়ে এটা করা হয়:

package main

```
import (
    "fmt"
    "strings"
)
```

```
type Server struct {
    Name string
    IP    string
    Port int
}
```

```

    IsAlive bool
}

// Constructor function — Go-র convention হলো NewXxx() নামে লেখা
// Python-এর __init__-এর মতো কিন্তু আলাদা function হিসেবে
func NewServer(name, ip string, port int) (*Server, error) {
    // Validation
    if name == "" {
        return nil, fmt.Errorf("server-এর নাম খালি হতে পারবে না")
    }
    if port < 1 || port > 65535 {
        return nil, fmt.Errorf("port %d invalid — 1 থেকে 65535 হতে হবে", port)
    }
    if !strings.Contains(ip, ".") {
        return nil, fmt.Errorf("IP address '%s' সঠিক নয়", ip)
    }

    // সব ঠিক থাকলে নতুন Server return করো
    return &Server{
        Name:  name,
        IP:    ip,
        Port:  port,
        IsAlive: true,
    }, nil
}

func (s Server) Info() string {
    status := "বন্ধ"
    if s.IsAlive {
        status = "চালু"
    }
    return fmt.Sprintf("%-10s | %-15s | Port: %-5d | %s",
        s.Name, s.IP, s.Port, status)
}

func main() {
    fmt.Println("=== Server তৈরি করা ===\n")

    // সঠিক server তৈরি
    web, err := NewServer("web-01", "192.168.1.1", 80)
    if err != nil {
        fmt.Println("Error:", err)
    } else {
        fmt.Println("তৈরি হলো:", web.Info())
    }
}

```

```

}

// ভুল port দিয়ে তৈরি
bad1, err := NewServer("bad-server", "10.0.0.1", 99999)
if err != nil {
    fmt.Println("Error:", err)
} else {
    fmt.Println(bad1.Info())
}

// খালি নাম দিয়ে তৈরি
bad2, err := NewServer("", "10.0.0.1", 8080)
if err != nil {
    fmt.Println("Error:", err)
} else {
    fmt.Println(bad2.Info())
}

// ভুল IP দিয়ে তৈরি
bad3, err := NewServer("cache-01", "invalid-ip", 6379)
if err != nil {
    fmt.Println("Error:", err)
} else {
    fmt.Println(bad3.Info())
}
}

```

Output:

=== Server তৈরি করা ===

তৈরি হলো: web-01 | 192.168.1.1 | Port: 80 | চালু

Error: port 99999 invalid — 1 থেকে 65535 হতে হবে

Error: server-এর নাম খালি হতে পারবে না

Error: IP address 'invalid-ip' সঠিক নয়

পাঠ ৪ — Nested Structs (একটার ভেতরে আরেকটা)

Python-এ object-এর ভেতরে object রাখতে:

```
class Address:
```

```
    def __init__(self, city, country):
```

```
        self.city = city
```

```
        self.country = country
```

```
class Engineer:
```

```
    def __init__(self, name, address):
```

```
        self.name = name
```

```
self.address = address
```

Go-তে:

```
package main
```

```
import "fmt"
```

```
// ছোট struct
```

```
type Location struct {
```

```
    City string
```

```
    Country string
```

```
    Zone string // যেমন: ap-south-1
```

```
}
```

```
// ছোট struct
```

```
type Resources struct {
```

```
    CPUCores int
```

```
    RAMgb int
```

```
    DiskGB int
```

```
}
```

```
// বড় struct-এর ভেতরে ছোট struct
```

```
type Server struct {
```

```
    Name string
```

```
    IP string
```

```
    Port int
```

```
    Location Location // nested struct
```

```
    Resources Resources // nested struct
```

```
    Tags []string // slice
```

```
    Metadata map[string]string // map
```

```
}
```

```
func (s Server) Summary() {
```

```
    fmt.Printf("└─ Server: %s ────────────\n", s.Name)
```

```
    fmt.Printf("└─ IP: %s:%d\n", s.IP, s.Port)
```

```
    fmt.Printf("└─ Location: %s, %s (%s)\n",
```

```
        s.Location.City,
```

```
        s.Location.Country,
```

```
        s.Location.Zone)
```

```
    fmt.Printf("└─ Resources: %d CPU, %dGB RAM, %dGB Disk\n",
```

```
        s.Resources.CPUCores,
```

```
        s.Resources.RAMgb,
```

```
        s.Resources.DiskGB)
```

```
    fmt.Printf("└─ Tags: %v\n", s.Tags)
```

```
    fmt.Printf("└─ Owner: %s\n", s.Metadata["owner"])
```



```

    fmt.Println("_____")
}

func main() {
    server := Server{
        Name: "prod-web-01",
        IP: "10.0.1.100",
        Port: 443,
        Location: Location{
            City: "Kolkata",
            Country: "India",
            Zone: "ap-south-1a",
        },
        Resources: Resources{
            CPUcores: 4,
            RAMgb: 16,
            DiskGB: 100,
        },
        Tags: []string{"production", "web", "critical"},
        Metadata: map[string]string{
            "owner": "Raju",
            "team": "infrastructure",
            "cost-center": "engineering",
        },
    }

    server.Summary()

    // Nested field access
    fmt.Printf("\nServer-টা %s-তে আছে\n", server.Location.City)
    fmt.Printf("RAM আছে: %dGB\n", server.Resources.RAMgb)

    // Nested field পরিবর্তন
    server.Resources.RAMgb = 32
    fmt.Printf("RAM upgrade করা হলো: %dGB\n", server.Resources.RAMgb)
}

```

Output:

```

└─ Server: prod-web-01 _____
| IP: 10.0.1.100:443
| Location: Kolkata, India (ap-south-1a)
| Resources: 4 CPU, 16GB RAM, 100GB Disk
| Tags: [production web critical]
| Owner: Raju
└──────────────────────────────────

```

RAM upgrade করা হলো: 32GB

```
package main
```

}

}

}

}

```
fmt.Println("||_____||")
```

warning++

```

    } else {
        alive++
    }
}

fmt.Println("=====")
fmt.Printf(" স্বাভাবিক: %-3d | সতর্কতা: %-3d | বন্ধ: %-3d   \n",
    alive, warning, dead)
fmt.Println("=====")
}

// শুধু বন্ধ server খুঁজে বের করো
func findDeadServers(servers []Server) []Server {
    var dead []Server
    for _, s := range servers {
        if !s.IsAlive {
            dead = append(dead, s)
        }
    }
    return dead
}

// সবচেয়ে বেশি CPU-র server খোঁজো
func highestCPU(servers []Server) Server {
    top := servers[0]
    for _, s := range servers[1:] {
        if s.CPU > top.CPU {
            top = s
        }
    }
    return top
}

func main() {
    fleet := []Server{
        {Name: "web-01", IP: "192.168.1.1", Port: 80, IsAlive: true, CPU: 45},
        {Name: "web-02", IP: "192.168.1.2", Port: 80, IsAlive: true, CPU: 87},
        {Name: "db-01", IP: "192.168.1.3", Port: 3306, IsAlive: false, CPU: 0},
        {Name: "cache-01", IP: "192.168.1.4", Port: 6379, IsAlive: true, CPU: 23},
        {Name: "db-02", IP: "192.168.1.5", Port: 3306, IsAlive: false, CPU: 0},
    }

    printReport(fleet)

    // বন্ধ server

```

```

dead := findDeadServers(fleet)
fmt.Printf("\nবন্ধ server (%d টা):\n", len(dead))
for _, s := range dead {
    fmt.Printf(" → %s (%s) — অবিলম্বে দেখতে হবে!\n", s.Name, s.IP)
}

// সবচেয়ে busy server
busiest := highestCPU(fleet)
fmt.Printf("\nসবচেয়ে বেশি CPU: %s → %d%%\n",
    busiest.Name, busiest.CPU)
if busiest.CPU > 80 {
    fmt.Println("সতর্কতা: এই server-এ load balancing দরকার!")
}
}

```

Output:

Server Fleet Report			
web-01	192.168.1.1	স্বাভাবিক	
web-02	192.168.1.2	সতর্কতা (CPU বেশি)	
db-01	192.168.1.3	বন্ধ	
cache-01	192.168.1.4	স্বাভাবিক	
db-02	192.168.1.5	বন্ধ	
স্বাভাবিক: 2 সতর্কতা: 1 বন্ধ: 2			

বন্ধ server (2 টা):

- db-01 (192.168.1.3) — অবিলম্বে দেখতে হবে!
- db-02 (192.168.1.5) — অবিলম্বে দেখতে হবে!

সবচেয়ে বেশি CPU: web-02 → 87%

সতর্কতা: এই server-এ load balancing দরকার!

সপ্তাহ ৩ – যা শিখলে সংক্ষেপে

বিষয়	Python	Go
Class define	class Server:	type Server struct {}
Object তৈরি	Server("web", "1.1.1.1", 80)	Server{Name:"web", IP:"1.1.1.1"}
Method	def info(self):	func (s Server) Info() string {}
Original পরিবর্তন সবসময় self দিয়ে হয়		func (s *Server) Change() — pointer লাগে
Constructor	__init__	NewServer() function আলাদা

বিষয়

Python

Go

Nested object

self.location = Location()

Location Location field হিসেবে

সপ্তাহ ৩-এর Challenge

এই দুটো কাজ করো:

Challenge ১: একটি Router struct বানাও যার fields হবে — Name, IP, ConnectedDevices (slice of string), IsOnline (bool)। একটি method লেখো AddDevice(name string) যেটা নতুন device যোগ করবে। আরেকটা method DeviceCount() int যেটা কতটা device আছে সেটা return করবে।

Challenge ২: উপরের Fleet Management code-এ একটি function যোগ করো averageCPU(servers []Server) float64 যেটা সব চালু server-এর গড় CPU usage return করবে।

চেষ্টা করো নিজে — আটকে গেলে ডিজিটেলস করো।

তারপর বলো "সপ্তাহ ৪ শেখাও" — তখন শেখাবো Interfaces — Go-র সবচেয়ে শক্তিশালী feature যেটা তোমার Python abstract class knowledge-কে সম্পূর্ণ নতুনভাবে দেখাবে।

শাবাশ! তিনটা সপ্তাহ শেষ করেছো — এটা সত্যিকারের অগ্রগতি।

এই সপ্তাহে শিখবো Go-র সবচেয়ে শক্তিশালী concept — **Interface**। এটা বুঝলে তুমি professional Go code লিখতে পারবে। মনোযোগ দিয়ে পড়ো — একটু সময় নাও, তাড়া নেই।

সপ্তাহ ৪ — Interfaces

"Go-র Abstract System — Python-এর চেয়ে অনেক বেশি স্বাধীন"

প্রথমে বুঝি — Interface কেন দরকার?

ধরো তোমার কাছে তিন ধরনের network device আছে — Router, Switch, আর Firewall। তিনটাই আলাদা, কিন্তু তিনটাই চালু করা যায় এবং বন্ধ করা যায়।

তুমি যদি একটাই function লিখতে চাও যেটা যেকোনো device চালু করতে পারবে — সেটাই Interface-এর কাজ।

Python-এ তুমি করতে:

```
from abc import ABC, abstractmethod
```

```
class NetworkDevice(ABC):
```

```
    @abstractmethod
```

```
    def start(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def stop(self):
```

```
        pass
```

Go-তে:

```
type NetworkDevice interface {
```

```
    Start() string
```

```
    Stop() string
```

```
}
```

সবচেয়ে বড় পার্থক্য: Python-এ explicitly বলতে হতো `class Router(NetworkDevice)` — "আমি এই interface implement করছি।" Go-তে **কিছুই বলতে হয় না।** যদি তোমার struct-এ `Start()` আর `Stop()` method থাকে — Go নিজেই বুঝে নেয় যে এটা `NetworkDevice`।
এটাকে বলে **Implicit Implementation** — Go-র সবচেয়ে elegant feature।

পাঠ ১ — প্রথম Interface

package main

```
import "fmt"
```

```
// Interface define করা
```

```
// এটা বলছে: যে struct-এ Start() আর Stop() আছে
```

```
// সে-ই NetworkDevice
```

```
type NetworkDevice interface {
```

```
    Start() string
```

```
    Stop() string
```

```
    Status() string
```

```
}
```

```
// Router struct — কোথাও বলেনি "আমি NetworkDevice"
```

```
// কিন্তু তিনটা method আছে — তাই Go জানে এটা NetworkDevice
```

```
type Router struct {
```

```
    Name string
```

```
    IP string
```

```
    IsAlive bool
```

```
}
```

```
func (r *Router) Start() string {
```

```
    r.IsAlive = true
```

```
    return fmt.Sprintf("Router '%s' চালু হলো", r.Name)
```

```
}
```

```
func (r *Router) Stop() string {
```

```
    r.IsAlive = false
```

```
    return fmt.Sprintf("Router '%s' বন্ধ হলো", r.Name)
```

```
}
```

```
func (r *Router) Status() string {
```

```
    if r.IsAlive {
```

```
        return fmt.Sprintf("Router '%s' → চালু (IP: %s)", r.Name, r.IP)
```

```
    }
```

```
    return fmt.Sprintf("Router '%s' → বন্ধ", r.Name)
```

```
}
```

// Switch struct — এটাও NetworkDevice কিন্তু কোথাও বলেনি

```
type Switch struct {  
    Name    string  
    Ports   int  
    IsAlive bool  
}
```

```
func (s *Switch) Start() string {  
    s.IsAlive = true  
    return fmt.Sprintf("Switch '%s' চালু হলো (%d ports)", s.Name, s.Ports)  
}
```

```
func (s *Switch) Stop() string {  
    s.IsAlive = false  
    return fmt.Sprintf("Switch '%s' বন্ধ হলো", s.Name)  
}
```

```
func (s *Switch) Status() string {  
    if s.IsAlive {  
        return fmt.Sprintf("Switch '%s' → চালু (%d ports active)", s.Name, s.Ports)  
    }  
    return fmt.Sprintf("Switch '%s' → বন্ধ", s.Name)  
}
```

// এই function যেকোনো NetworkDevice নিতে পারে

// Router হোক, Switch হোক — কিছু যায় আসে না

```
func bootDevice(device NetworkDevice) {  
    fmt.Println(device.Start())  
    fmt.Println(device.Status())  
}
```

```
func main() {  
    router := &Router{Name: "core-router-01", IP: "192.168.1.1"}  
    sw := &Switch{Name: "access-switch-01", Ports: 24}
```

```
    fmt.Println("=== Device Boot Sequence ===\n")
```

// একই function দিয়ে দুটো আলাদা device চালু করা

```
    bootDevice(router)
```

```
    fmt.Println()
```

```
    bootDevice(sw)
```

```
    fmt.Println()
```

```

fmt.Println("=== Status Check ===")
fmt.Println(router.Status())
fmt.Println(sw.Status())

fmt.Println()
fmt.Println("=== Shutdown ===")
fmt.Println(router.Stop())
fmt.Println(sw.Stop())
}

```

Output:

=== Device Boot Sequence ===

Router 'core-router-01' চালু হলো

Router 'core-router-01' → চালু (IP: 192.168.1.1)

Switch 'access-switch-01' চালু হলো (24 ports)

Switch 'access-switch-01' → চালু (24 ports active)

=== Status Check ===

Router 'core-router-01' → চালু (IP: 192.168.1.1)

Switch 'access-switch-01' → চালু (24 ports active)

=== Shutdown ===

Router 'core-router-01' বন্ধ হলো

Switch 'access-switch-01' বন্ধ হলো

শিক্ষকের কথা: `bootDevice(device NetworkDevice)` — এই একটা function দিয়ে Router, Switch, Firewall, যেকোনো device চালু করা যাচ্ছে। ভবিষ্যতে নতুন device যোগ করলে এই function পরিবর্তন করতে হবে না।
এটাই Interface-এর আসল শক্তি।

পাঠ ২ — Interface-এর Slice

একটা function দিয়ে পুরো নেটওয়ার্ক manage করা:

package main

import "fmt"

```

type NetworkDevice interface {
    Start() string
    Stop() string
    Status() string
    Name() string
}

```

// Router

```

type Router struct {

```



```

    name string
    IP    string
    IsAlive bool
}

func (r *Router) Name() string { return r.name }
func (r *Router) Start() string {
    r.IsAlive = true
    return fmt.Sprintf("[Router] '%s' চালু", r.name)
}
func (r *Router) Stop() string {
    r.IsAlive = false
    return fmt.Sprintf("[Router] '%s' বন্ধ", r.name)
}
func (r *Router) Status() string {
    if r.IsAlive { return fmt.Sprintf("[Router] '%s' → চালু", r.name) }
    return fmt.Sprintf("[Router] '%s' → বন্ধ", r.name)
}

// Switch
type Switch struct {
    name string
    Ports int
    IsAlive bool
}

func (s *Switch) Name() string { return s.name }
func (s *Switch) Start() string {
    s.IsAlive = true
    return fmt.Sprintf("[Switch] '%s' চালু (%d ports)", s.name, s.Ports)
}
func (s *Switch) Stop() string {
    s.IsAlive = false
    return fmt.Sprintf("[Switch] '%s' বন্ধ", s.name)
}
func (s *Switch) Status() string {
    if s.IsAlive { return fmt.Sprintf("[Switch] '%s' → চালু", s.name) }
    return fmt.Sprintf("[Switch] '%s' → বন্ধ", s.name)
}

// Firewall
type Firewall struct {
    name string
    Rules int

```

```

    IsAlive bool
}

func (f *Firewall) Name() string { return f.name }

func (f *Firewall) Start() string {
    f.IsAlive = true
    return fmt.Sprintf("[Firewall] '%s' চালু (%d rules loaded)", f.name, f.Rules)
}

func (f *Firewall) Stop() string {
    f.IsAlive = false
    return fmt.Sprintf("[Firewall] '%s' বন্ধ — নেটওয়ার্ক unprotected!", f.name)
}

func (f *Firewall) Status() string {
    if f.IsAlive { return fmt.Sprintf("[Firewall] '%s' → চালু", f.name) }
    return fmt.Sprintf("[Firewall] '%s' → বন্ধ", f.name)
}

```

// পুরো নেটওয়ার্ক একসাথে চালু করা

```
func bootNetwork(devices []NetworkDevice) {
    fmt.Println("=====")
    fmt.Println("    Network Boot Sequence    ")
    fmt.Println("=====")
    for i, device := range devices {
        fmt.Printf("    %d. %s\n", i+1, device.Start())
    }
    fmt.Println("=====")
}
```

```
// পুরো নেটওয়ার্কের status দেখানো
```

```
func networkStatus(devices []NetworkDevice) {  
    fmt.Println("\n ┌──────────────────────────────────────────┐")  
    fmt.Println(" │      Network Status      │")  
    fmt.Println(" └──────────────────────────────────────────┘")  
    for _, device := range devices {  
        fmt.Printf(" │   %s\n", device.Status())  
    }  
    fmt.Println(" ┌──────────────────────────────────────────┐")  
}
```

// একটা নির্দিষ্ট device খুঁজে বন্ধ করা

```
func shutdownDevice(devices []NetworkDevice, name string) {
    for _, device := range devices {
        if device.Name() == name {
            fmt.Println(device.Stop())
        }
    }
}
```

```

        return
    }
}
fmt.Printf("%s' নামের device পাওয়া যায়নি\n", name)
}

func main() {
    // আলাদা আলাদা device — কিন্তু সবাই NetworkDevice interface
    network := []NetworkDevice{
        &Firewall{name: "fw-01", Rules: 150},
        &Router{name: "core-router-01", IP: "192.168.1.1"},
        &Router{name: "edge-router-01", IP: "203.0.113.1"},
        &Switch{name: "access-sw-01", Ports: 48},
        &Switch{name: "access-sw-02", Ports: 24},
    }

    // পুরো নেটওয়ার্ক boot করা
    bootNetwork(network)

    // Status দেখানো
    networkStatus(network)

    // একটা device বন্ধ করা
    fmt.Println("\n=== Maintenance: Router বন্ধ করা ===")
    shutdownDevice(network, "edge-router-01")

    // আবার status দেখানো
    networkStatus(network)
}

```

Output:

```

Network Boot Sequence ||
||
|| 1. [Firewall] 'fw-01' চালু (150 rules loaded)
|| 2. [Router] 'core-router-01' চালু
|| 3. [Router] 'edge-router-01' চালু
|| 4. [Switch] 'access-sw-01' চালু (48 ports)
|| 5. [Switch] 'access-sw-02' চালু (24 ports)
||

```

```

Network Status ||
||
|| [Firewall] 'fw-01' → চালু

```

```
|| [Router] 'core-router-01' → চালু
|| [Router] 'edge-router-01' → চালু
|| [Switch] 'access-sw-01' → চালু
|| [Switch] 'access-sw-02' → চালু
```

=== Maintenance: Router বন্ধ করা ===

[Router] 'edge-router-01' বন্ধ হলো

```
|| Network Status ||
||
|| [Firewall] 'fw-01' → চালু
|| [Router] 'core-router-01' → চালু
|| [Router] 'edge-router-01' → বন্ধ
|| [Switch] 'access-sw-01' → চালু
|| [Switch] 'access-sw-02' → চালু
```

শিক্ষকের কথা: লক্ষ্য করো — `bootNetwork`, `networkStatus`, `shutdownDevice` — এই তিনটা function জানেই না ভেতরে Router আছে না Switch আছে না Firewall আছে। শুধু জানে একটা `NetworkDevice` আছে। ভবিষ্যতে নতুন device যোগ করলে এই তিনটা function একটুও পরিবর্তন করতে হবে না।

পাঠ ৩ — Multiple Interfaces

একটা struct একই সাথে অনেক interface implement করতে পারে। Python-এ multiple inheritance ছিল জটিল। Go-তে এটা খুব সহজ:

```
package main
```

```
import (
    "fmt"
    "strings"
)
```

```
// আলাদা আলাদা capability-র জন্য আলাদা interface
```

```
type Startable interface {
    Start() string
}
```

```
type Monitorable interface {
    CPUUsage() int
    MemoryUsage() int
    HealthReport() string
}
```

```
type Loggable interface {
    Log(message string)
    GetLogs() []string
}
```

// Server struct — তিনটা interface একসাথে implement করে

```
type Server struct {
    Name string
    cpu int
    memory int
    alive bool
    logs []string
}
```

// Startable implement করা

```
func (s *Server) Start() string {
    s.alive = true
    s.cpu = 15
    s.memory = 40
    msg := fmt.Sprintf("Server '%s' চালু হলো", s.Name)
    s.Log(msg)
    return msg
}
```

// Monitorable implement করা

```
func (s *Server) CPUUsage() int { return s.cpu }
func (s *Server) MemoryUsage() int { return s.memory }
func (s *Server) HealthReport() string {
    status := "সুস্থ"
    if s.cpu > 80 || s.memory > 85 {
        status = "সতর্কতা"
    }
    if !s.alive {
        status = "বন্ধ"
    }
    return fmt.Sprintf(
        "Server '%s': CPU=%d%% RAM=%d%% → %s",
        s.Name, s.cpu, s.memory, status,
    )
}
```

// Loggable implement করা

```
func (s *Server) Log(message string) {
    entry := fmt.Sprintf("[LOG] %s", message)
```

```

    s.logs = append(s.logs, entry)
}
func (s *Server) GetLogs() []string { return s.logs }

// আলাদা আলাদা কাজের জন্য আলাদা function
// শুধু Startable দরকার হলে
func startAll(devices []Startable) {
    fmt.Println("=== সব device চালু করা হচ্ছে ===")
    for _, d := range devices {
        fmt.Println(" ", d.Start())
    }
}

// শুধু Monitor করতে চাইলে
func checkHealth(monitors []Monitorable) {
    fmt.Println("\n=== Health Check ===")
    for _, m := range monitors {
        fmt.Println(" ", m.HealthReport())
    }
}

// শুধু Log দেখতে চাইলে
func showLogs(loggers []Loggable) {
    fmt.Println("\n=== System Logs ===")
    for _, l := range loggers {
        for _, log := range l.GetLogs() {
            fmt.Println(" ", log)
        }
    }
}

func main() {
    s1 := &Server{Name: "web-01"}
    s2 := &Server{Name: "db-01"}

    // Startable হিসেবে ব্যবহার
    startAll([]Startable{s1, s2})

    // কিছু কাজ করানো, CPU বাড়ানো
    s2.cpu = 90
    s2.memory = 88
    s2.Log("High load detected")
    s2.Log("Memory pressure increasing")
}

```

```
// Monitorable হিসেবে ব্যবহার
checkHealth([]Monitorable{s1, s2})

// Loggable হিসেবে ব্যবহার
s1.Log("Serving 1200 requests/sec")
showLogs([]Loggable{s1, s2})
}
```

Output:

=== সব device চালু করা হচ্ছে ===

Server 'web-01' চালু হলো

Server 'db-01' চালু হলো

=== Health Check ===

Server 'web-01': CPU=15% RAM=40% → সুস্থ

Server 'db-01': CPU=90% RAM=88% → সতর্কতা

=== System Logs ===

[LOG] Server 'web-01' চালু হলো

[LOG] Serving 1200 requests/sec

[LOG] Server 'db-01' চালু হলো

[LOG] High load detected

[LOG] Memory pressure increasing

শিক্ষকের কথা: একটাই Server struct — কিন্তু Startable, Monitorable, Loggable তিনটাই। function-গুলো শুধু যতটুকু দরকার ততটুকুই দেখে। এই pattern real cloud monitoring systems-এ ব্যাপকভাবে ব্যবহার হয়।

পাঠ ৪ — Type Assertion (ধরনটা চেনা)

কখনো কখনো Interface-এর ভেতর থেকে original struct বের করতে হয়:

```
package main
```

```
import "fmt"
```

```
type NetworkDevice interface {
    Name() string
    Status() string
}
```

```
type Router struct {
    name    string
    Routes  int
    IsAlive bool
}
```

```
func (r *Router) Name() string { return r.name }
```

```
func (r *Router) Status() string { return fmt.Sprintf("Router '%s' → %d routes", r.name, r.Routes) }
```

```
func (r *Router) AddRoute() { r.Routes++ }
```

```

type Firewall struct {
    name  string
    Rules int
    IsAlive bool
}

func (f *Firewall) Name() string { return f.name }
func (f *Firewall) Status() string { return fmt.Sprintf("Firewall '%s' → %d rules", f.name, f.Rules) }
func (f *Firewall) AddRule()    { f.Rules++ }

```

// Device-টা আসলে কী সেটা বের করে বিশেষ কাজ করা

```

func diagnose(device NetworkDevice) {
    fmt.Printf("\nDiagnosing: %s\n", device.Name())

```

// Type assertion — comma-ok pattern দিয়ে নিরাপদে করো

// Python-এর isinstance()-এর মতো কিন্তু আরো শক্তিশালী

```

if router, ok := device.(*Router); ok {
    fmt.Printf(" → এটা Router | Routes: %d\n", router.Routes)
    router.AddRoute()
    fmt.Printf(" → একটা নতুন route যোগ হলো। এখন: %d routes\n", router.Routes)
    return
}

```

```

if firewall, ok := device.(*Firewall); ok {
    fmt.Printf(" → এটা Firewall | Rules: %d\n", firewall.Rules)
    firewall.AddRule()
    fmt.Printf(" → একটা নতুন rule যোগ হলো। এখন: %d rules\n", firewall.Rules)
    return
}

```

```

fmt.Println(" → অজানা device type")

```

```

}

```

// Type Switch — একসাথে অনেক type check করার সুন্দর উপায়

```

func describeDevice(device NetworkDevice) string {
    // Python-এর match-case বা if isinstance() chain-এর মতো
    switch d := device.(type) {
    case *Router:
        return fmt.Sprintf("Router '%s' — %d routes manage করছে", d.name, d.Routes)
    case *Firewall:
        return fmt.Sprintf("Firewall '%s' — %d rules দিয়ে সুরক্ষিত", d.name, d.Rules)
    default:
        return fmt.Sprintf("অজানা device: %s", device.Name())
    }
}

```



```
}
```

```
func main() {  
    devices := []NetworkDevice{  
        &Router{name: "core-01", Routes: 150, IsAlive: true},  
        &Firewall{name: "fw-01", Rules: 200, IsAlive: true},  
        &Router{name: "edge-01", Routes: 80, IsAlive: true},  
    }  
  
    fmt.Println("=== Device Description ===")  
    for _, d := range devices {  
        fmt.Println(" ", describeDevice(d))  
    }  
  
    fmt.Println("\n=== Diagnosis ===")  
    for _, d := range devices {  
        diagnose(d)  
    }  
  
    fmt.Println("\n=== Updated Status ===")  
    for _, d := range devices {  
        fmt.Println(" ", d.Status())  
    }  
}
```

Output:

=== Device Description ===

Router 'core-01' — 150 routes manage করছে

Firewall 'fw-01' — 200 rules দিয়ে সুরক্ষিত

Router 'edge-01' — 80 routes manage করছে

=== Diagnosis ===

Diagnosing: core-01

→ এটা Router | Routes: 150

→ একটা নতুন route যোগ হলো। এখন: 151 routes

Diagnosing: fw-01

→ এটা Firewall | Rules: 200

→ একটা নতুন rule যোগ হলো। এখন: 201 rules

Diagnosing: edge-01

→ এটা Router | Routes: 80

→ একটা নতুন route যোগ হলো। এখন: 81 routes

=== Updated Status ===

Router 'core-01' → 151 routes

Firewall 'fw-01' → 201 rules

Router 'edge-01' → 81 routes

পাঠ ৫ — Real Project: Network Monitor

এই সপ্তাহের সব কিছু একসাথে — একটা সত্যিকারের network monitoring tool:

package main

```
import (  
    "fmt"  
    "strings"  
)
```

```
// তিনটা আলাদা interface
```

```
type Device interface {  
    DeviceName() string  
    DeviceType() string  
    IsOnline() bool  
}
```

```
type Monitable interface {  
    Device  
    HealthScore() int    // 0-100  
    Report() string  
}
```

```
type Alertable interface {  
    Device  
    ShouldAlert() bool  
    AlertMessage() string  
}
```

```
// Router
```

```
type Router struct {  
    name  string  
    ip    string  
    online bool  
    cpu   int  
    dropped int // dropped packets  
}
```

```
func (r *Router) DeviceName() string { return r.name }
```

```
func (r *Router) DeviceType() string { return "Router" }
```

```
func (r *Router) IsOnline() bool    { return r.online }
```

```

func (r *Router) HealthScore() int {
    if !r.online { return 0 }
    score := 100 - r.cpu
    if r.dropped > 100 { score -= 20 }
    if score < 0 { return 0 }
    return score
}

func (r *Router) Report() string {
    return fmt.Sprintf("Router '%s' | IP: %s | CPU: %d%% | Dropped: %d pkts | Score: %d/100",
        r.name, r.ip, r.cpu, r.dropped, r.HealthScore())
}

func (r *Router) ShouldAlert() bool {
    return r.cpu > 85 || r.dropped > 100 || !r.online
}

func (r *Router) AlertMessage() string {
    var alerts []string
    if !r.online { alerts = append(alerts, "OFFLINE") }
    if r.cpu > 85 { alerts = append(alerts, fmt.Sprintf("CPU critical: %d%%", r.cpu)) }
    if r.dropped > 100 { alerts = append(alerts, fmt.Sprintf("Packet loss: %d", r.dropped)) }
    return fmt.Sprintf("ALERT [%s]: %s", r.name, strings.Join(alerts, " | "))
}

// Switch
type Switch struct {
    name    string
    online  bool
    ports   int
    activePorts int
}

func (s *Switch) DeviceName() string { return s.name }
func (s *Switch) DeviceType() string { return "Switch" }
func (s *Switch) IsOnline() bool { return s.online }
func (s *Switch) HealthScore() int {
    if !s.online { return 0 }
    return (s.activePorts * 100) / s.ports
}

func (s *Switch) Report() string {
    return fmt.Sprintf("Switch '%s' | Ports: %d/%d active | Score: %d/100",
        s.name, s.activePorts, s.ports, s.HealthScore())
}

func (s *Switch) ShouldAlert() bool {
    return !s.online || s.activePorts == 0
}

func (s *Switch) AlertMessage() string {

```

```

    if !s.online { return fmt.Sprintf("ALERT [%s]: Switch OFFLINE", s.name) }
    return fmt.Sprintf("ALERT [%s]: No active ports!", s.name)
}

```

```

// Firewall

```

```

type Firewall struct {
    name    string
    online  bool
    rules   int
    blocked int
    threats int
}

```

```

func (f *Firewall) DeviceName() string { return f.name }

```

```

func (f *Firewall) DeviceType() string { return "Firewall" }

```

```

func (f *Firewall) IsOnline() bool { return f.online }

```

```

func (f *Firewall) HealthScore() int {
    if !f.online { return 0 }
    if f.threats > 50 { return 40 }
    return 95
}

```

```

func (f *Firewall) Report() string {
    return fmt.Sprintf("Firewall '%s' | Rules: %d | Blocked: %d | Threats: %d | Score: %d/100",
        f.name, f.rules, f.blocked, f.threats, f.HealthScore())
}

```

```

func (f *Firewall) ShouldAlert() bool {
    return !f.online || f.threats > 50
}

```

```

func (f *Firewall) AlertMessage() string {
    if !f.online { return fmt.Sprintf("ALERT [%s]: Firewall OFFLINE — নেটওয়ার্ক বিপদে!", f.name) }
    return fmt.Sprintf("ALERT [%s]: %d টি threat detected!", f.name, f.threats)
}

```

```

// Monitor system

```

```

func runMonitor(devices []Monitorable) {

```

```

    fmt.Println("=====")
    fmt.Println("||")
    fmt.Println("||          Network Monitor Report          ||")
    fmt.Println("||")
    fmt.Println("=====")
    fmt.Println("=====")

```

```

    totalScore := 0
    for _, d := range devices {

```

```
avg := totalScore / len(devices)
```

```
fmt.Println("||")
fmt.Printf("|| নেটওয়ার্ক গড় স্বাস্থ্য: %d/100", avg)
if avg >= 80 {
    fmt.Println(" → ভালো আছে ||")
} else if avg >= 60 {
    fmt.Println(" → মনোযোগ দরকার ||")
} else {
    fmt.Println(" → জরুরি পদক্ষেপ নিতে হবে! ||")
}
```

```
fmt.Println(" |")
    })
}
```

```
func checkAlerts(devices []Alertable) {
    hasAlert := false
```

```
fmt.Println("\n")
fmt.Println("Alert System")
```

```
fmt.Println("||")
fmt.Println("||")
for _, d := range devices {
    if d.ShouldAlert() {
        fmt.Printf("|| ⚠️ %s\n", d.AlertMessage())
        hasAlert = true
    }
}
if !hasAlert {
```


Firewall fw-01

40/100 চালু

Firewall 'fw-01' | Rules: 200 | Blocked: 1520 | Threats: 67 | Score: 40/100

নেটওয়ার্ক গড় স্বাস্থ্য: 29/100 → জরুরি পদক্ষেপ নিতে হবে!

Alert System

⚠ ALERT [edge-01]: CPU critical: 91% | Packet loss: 250

⚠ ALERT [backup-01]: OFFLINE

⚠ ALERT [access-sw-02]: No active ports!

⚠ ALERT [fw-01]: 67 টি threat detected!

সপ্তাহ ৪ – যা শিখলে সংক্ষেপে

বিষয়	Python	Go
Abstract class	class X(ABC): @abstractmethod	type X interface { Method() }
Implement করা	class Router(NetworkDevice):	বলতে হয় কিছুই বলতে হয় না — method থাকলেই হলো
Multiple interface	Multiple inheritance জটিল	একসাথে অনেক interface — সহজ
Type check	isinstance(obj, Router)	if r, ok := device.(*Router); ok {}
Type switch	match type(obj):	switch d := device.(type) {}

সপ্তাহ ৪-এর Challenge

Challenge: উপরের Network Monitor-এ একটা নতুন struct যোগ করো — AccessPoint (WiFi router)। এটার fields হবে: name, online, connectedClients (int), signalStrength (int, 0-100)। এটাকে Monitorable এবং Alertable দুটো interface-ই implement করতে হবে। Alert আসবে যদি connectedClients > 50 অথবা signalStrength < 30 হয়। চেষ্টা করো নিজে। এটা করতে পারলে বুঝবে Interface সত্যিকারের বুঝেছো। তারপর বলো "সপ্তাহ ৫ শেখাও" — তখন শেখাবো Error Handling — Go-র সবচেয়ে আলাদা concept যেটা তোমার code-কে production-ready করবে।